

Properties of Regular Languages

1. Non-Regular Languages and Proof Techniques

Introduction to Non-Regular Languages

Not all languages can be recognized by finite automata. Some languages require unlimited memory to track information, which finite automata cannot provide since they have a fixed, finite number of states.

When is a Language Regular?

A language L is **regular** if and only if we can construct one of the following:

- A DFA (Deterministic Finite Automaton)
- An NFA (Non-deterministic Finite Automaton)
- An ϵ -NFA (Epsilon-NFA)
- A Regular Expression

When is a Language NOT Regular?

A language is not regular if we can prove that no finite automaton can be constructed for it.

Example: A Classic Non-Regular Language

Language $L = \{0^n 1^n \mid n \geq 0\}$

This language contains strings with equal numbers of 0s followed by equal numbers of 1s.

- Valid strings: ϵ , 01, 0011, 000111, 00001111
- Invalid strings: 0, 1, 001, 0111, 00011

Hypothesis: L is not regular.

Intuitive Rationale: To recognize this language, we need to "count" the number of 0s and then verify that exactly the same number of 1s follow. But a finite automaton has only a fixed number of states and cannot count arbitrarily large numbers.

More Formal Rationale (by Contradiction):

1. Assume L is regular (for contradiction)
2. Then there must exist a DFA for L
3. Let k = number of states in that DFA
4. Consider the string $w = 0^k 1^k$ (k zeros followed by k ones)
5. Clearly $w \in L$

Now, trace what happens as the DFA reads the first k symbols (all 0s):

- The DFA visits states $p_0, p_1, p_2, \dots, p_k$
- That's $k+1$ states visited
- But the DFA has only k states total!
- By the **Pigeonhole Principle**, at least one state must repeat
- Let $p_i = p_j$ where $i < j$ (the repeating state)

The Key Insight: Since there's a loop (from p_i back to $p_j = p_i$), we can:

- Skip the loop by reading only $0^{(k-(j-i))}$ instead of 0^k
- The remaining 1^k would still be accepted
- But $0^{(k-(j-i))}1^k$ has unequal numbers of 0s and 1s
- This string should NOT be in L , yet the DFA accepts it!

Contradiction! Therefore, our assumption was wrong, and L is not regular.

2. The Pumping Lemma for Regular Languages

What is the Pumping Lemma?

The Pumping Lemma is a formal technique used to prove that a given language is NOT regular. It provides a property that ALL regular languages must satisfy.

Statement of the Pumping Lemma

Let L be a regular language.

Then there exists some constant N (called the **Pumping Lemma Constant**) such that:

For every string $w \in L$ where $|w| \geq N$, there exists a way to break w into three parts $w = xyz$ such that:

1. $y \neq \epsilon$ (y is not empty)
2. $|xy| \leq N$ (the first two parts together have length at most N)
3. **For all $k \geq 0$, $xy^kz \in L$** (we can "pump" y any number of times and stay in L)

Important Note: If a language is regular, it MUST satisfy this property. If we can show a language does NOT satisfy this property, then it CANNOT be regular.

Proof of the Pumping Lemma

Given: L is regular

Proof:

1. Since L is regular, it has a DFA
2. Let N = number of states in the DFA
3. Take any string $w \in L$ where $|w| \geq N$
4. Write $w = a_1a_2...a_m$ where $m \geq N$
5. As the DFA reads the first N symbols, it visits states: $p_0, p_1, p_2, \dots, p_n$
6. That's $N+1$ states visited, but only N states exist
7. By the **Pigeonhole Principle**, at least one state repeats: $p_i = p_j$ where $0 \leq i < j \leq N$

Breaking w into xyz :

- $x = a_1a_2...a_i$ (path from start to p_i)
- $y = a_{i+1}a_{i+2}...a_j$ (loop from p_i back to $p_j = p_i$)
- $z = a_{j+1}a_{j+2}...a_m$ (path from p_j to final state)

Verifying the three conditions:

1. $y \neq \epsilon$: Since $i < j$, y contains at least one symbol ✓
2. $|xy| \leq N$: The repetition occurs within the first N symbols ✓
3. $xy^kz \in L$ for all $k \geq 0$:
 - When $k = 0$: xz reaches the final state (skip the loop)
 - When $k > 0$: xy^kz loops k times and still reaches the final state ✓

Detailed Example 1: Equal Numbers of 0s and 1s

Language: $L_{eq} = \{w \mid w \text{ is a binary string with equal numbers of 0s and 1s}\}$

Examples of strings in L_{eq} : 01, 10, 0011, 0101, 1100, 001011

Claim: L_{eq} is not regular

Proof:

1. **Assume (by contradiction)** that L_{eq} is regular
2. **Then** the Pumping Lemma applies
3. **Let** N = the Pumping Lemma constant (adversary's choice)
4. **(Your choice)** Choose $w = 0^N1^N$ as the template string

- Note: $w \in L_{eq}$ because it has N zeros and N ones
 - Note: $|w| = 2N \geq N \checkmark$
5. **By Pumping Lemma**, w can be broken into $w = xyz$ where:
- $y \neq \epsilon$
 - $|xy| \leq N$
 - $xy^kz \in L_{eq}$ for all $k \geq 0$
6. **Key Observation:** Since $|xy| \leq N$, both x and y consist entirely of 0s
- This is because the first N symbols of w are all 0s
 - Therefore $y = 0^m$ for some $m \geq 1$ (since $y \neq \epsilon$)
7. **Breaking down the structure:**
- x contains at most $N-1$ zeros
 - y contains between 1 and N zeros (let's say m zeros)
 - z contains all N ones plus any remaining zeros
8. **(Pumping down with $k=0$):** Consider $xy^0z = xz$
- Number of 0s in $xz = (N - m)$ where $m \geq 1$
 - So xz has at most $N-1$ zeros
 - Number of 1s in $xz = N$
 - Therefore xz has unequal numbers of 0s and 1s
 - **Thus $xy^0z \notin L_{eq}$**
9. **This contradicts condition 3 of the Pumping Lemma!**

Conclusion: Our assumption was wrong. L_{eq} is NOT regular.

Alternative approach (pumping up): We could also use $k=2$:

- xy^2z would have $N + m$ zeros and N ones (unequal)
- Thus $xy^2z \notin L_{eq}$

Example 2: Practice Exercise

Language: $L = \{0^n 1 0^n \mid n \geq 1\}$

Examples: 010, 00100, 0001000, 000010000

Task: Prove L is not regular (try this yourself using the pumping lemma!)

Hint for solution:

1. Choose $w = 0^N 1 0^N$
2. Since $|xy| \leq N$, xy consists only of 0s before the 1
3. Pump down ($k=0$) or pump up ($k=2$)
4. Show that the number of 0s before and after the 1 become unequal

Example 3: Perfect Squares

Language: $L = \{0^i \mid i \text{ is a perfect square}\}$

Examples: ϵ ($0^2 = 0$), 0 (1^2), 0000 (2^2), 0000000000 (3^2), 000000000000000000 (4^2)

Claim: L is not regular

Proof:

1. **Assume** L is regular (for contradiction)
2. **Let** N = Pumping Lemma constant
3. **(Choose)** $w = 0^{(N^2)}$ (N squared zeros)
 - $w \in L$ because N^2 is a perfect square
 - $|w| = N^2 \geq N \checkmark$
4. **By Pumping Lemma**, $w = xyz$ where:
 - $y \neq \epsilon$
 - $|xy| \leq N$
 - $xy^k z \in L$ for all $k \geq 0$
5. **From conditions 1 and 2:** y contains between 1 and N zeros
6. **(Try $k=0$ first - pumping down):**
 - Number of zeros in $xy^0z = N^2 - |y|$
 - Since $1 \leq |y| \leq N$:
 - $N^2 - N \leq \#zeros(xy^0z) \leq N^2 - 1$

Is this a perfect square?

- The previous perfect square is $(N-1)^2 = N^2 - 2N + 1$
- We need to show: $(N-1)^2 < N^2 - N$
- Simplifying: $N^2 - 2N + 1 < N^2 - N$
- This gives: $-2N + 1 < -N$, which means: $1 < N$

Problem: If $N = 1$, then $(N-1)^2 = 0$ and $N^2 - N = 0$, so they're equal!

- This means pumping down doesn't work for $N = 1$

7. (Try $k=2$ - pumping up):

- Number of zeros in $xy^2z = N^2 + |y|$
- Since $1 \leq |y| \leq N$:
- $N^2 + 1 \leq \#zeros(xy^2z) \leq N^2 + N$

Is this a perfect square?

- The next perfect square is $(N+1)^2 = N^2 + 2N + 1$
- We need: $N^2 + N < (N+1)^2$
- Simplifying: $N^2 + N < N^2 + 2N + 1$
- This gives: $0 < N + 1$

This is always true for $N > 0$!

- Therefore $N^2 < N^2 + 1 \leq \#zeros(xy^2z) \leq N^2 + N < (N+1)^2$
- So xy^2z has a number of zeros strictly between two consecutive perfect squares
- **Thus $xy^2z \notin L$**

8. Contradiction! Condition 3 of the Pumping Lemma is violated.

Conclusion: L is not regular. ■

Key Lesson: Sometimes pumping down ($k=0$) doesn't work for all values of N . In such cases, try pumping up ($k=2$ or higher).

3. Closure Properties of Regular Languages

What is a Closure Property?

A closure property states that when you combine regular languages using certain operations, the result is also a regular language.

Formal Definition: Regular languages are **closed** under an operation ★ if:

- Whenever L and M are regular languages,
- $L \star M$ is also a regular language

List of Closure Properties

Regular languages are closed under:

1. **Union** ($L \cup M$)
2. **Intersection** ($L \cap M$)
3. **Complementation** ($\bar{L}$)
4. **Difference** ($L - M$)
5. **Concatenation** (LM)
6. **Kleene Closure** (L^*)
7. **Reversal** (L^R)
8. **Homomorphism** ($h(L)$)
9. **Inverse Homomorphism** ($h^{-1}(L)$)

Let's prove each of these!

1. Closure Under Union

Theorem: If L and M are regular languages, then $L \cup M$ is also regular.

Proof Using Regular Expressions:

1. Since L and M are regular, they have regular expressions R and S respectively
2. The regular expression $R + S$ describes $L \cup M$
3. Since $R + S$ is a regular expression, $L \cup M$ is regular ✓

2. Closure Under Concatenation

Theorem: If L and M are regular languages, then LM is also regular.

Proof:

1. L has regular expression R , M has regular expression S
2. The regular expression RS describes LM
3. Therefore LM is regular ✓

3. Closure Under Kleene Closure

Theorem: If L is a regular language, then L^* is also regular.

Proof:

1. L has regular expression R
2. The regular expression R^* describes L^*
3. Therefore L^* is regular ✓

4. Closure Under Complementation

Theorem: If L is a regular language over alphabet Σ , then $\bar{L} = \Sigma^* - L$ is also regular.

Proof Using DFA Construction:

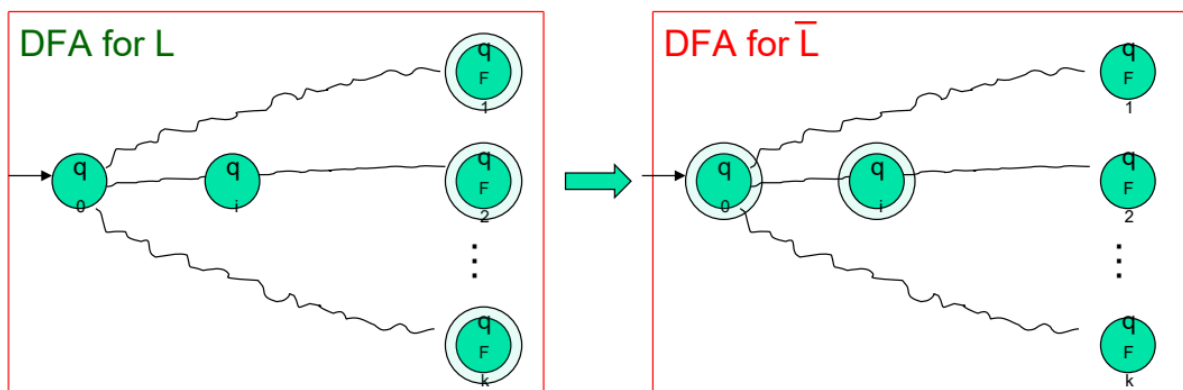
Given a DFA for L :

1. Take the DFA that recognizes L
2. **Swap** all final and non-final states:
 - Every final state becomes non-final
 - Every non-final state becomes final
3. The resulting DFA recognizes $\bar{L}$

Why This Works:

- A string w is in $\bar{L}$ if and only if w is NOT in L
- The original DFA rejects w if and only if it ends in a non-final state
- By swapping, these rejecting states become accepting states
- Therefore, the new DFA accepts exactly $\bar{L}$

Visual Example:



Important Note: This construction assumes q_0 (start state) is non-final. If the start state is final in the original DFA, simply reverse the transformation.

5. DFA Minimization and Equivalence

Why Minimize DFAs?

- Smaller DFAs are easier to understand
- Faster execution
- Less memory usage
- Canonical representation (unique minimal DFA for each language)

Key Concept: Equivalent States

Definition: Two states p and q in a DFA are **equivalent** (written $p \equiv q$) if and only if:

For every string w :

- Starting from p , w leads to acceptance $\leftrightarrow$ Starting from q , w leads to acceptance
- Starting from p , w leads to rejection $\leftrightarrow$ Starting from q , w leads to rejection

Important Principle: *“The past doesn’t matter — only the future does!”*

It does **not** matter how we reached states p or q .

What matters is whether they behave the same for **all future inputs**.

Visual Representation:

...past... $\rightarrow p \rightarrow w \rightarrow \text{accept/reject}$

...past... $\rightarrow q \rightarrow w \rightarrow \text{accept/reject}$

If both always give the same result $\rightarrow p \equiv q$

The Table-Filling Algorithm

This algorithm identifies which pairs of states are **distinguishable** (NOT equivalent).

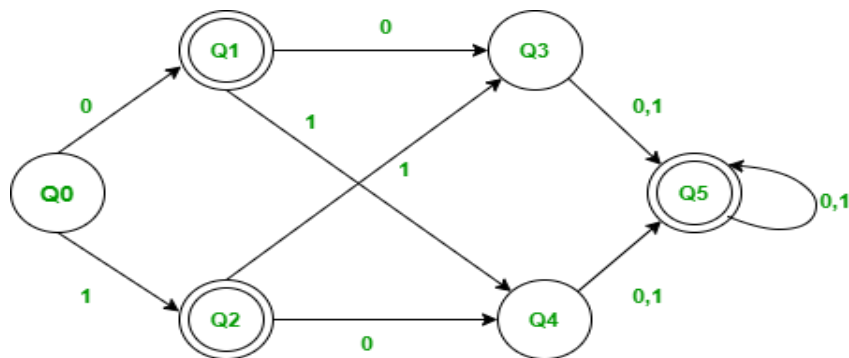
Algorithm Overview:

1. **Initialize:**
Mark all pairs of (*final*, *non-final*) states as distinguishable.
2. **Iterate:**
For each unmarked pair (p, q) :
 - For each input symbol a :
 - If $\delta(p, a)$ and $\delta(q, a)$ are already marked distinguishable,
 $\rightarrow$ mark (p, q) as distinguishable.

3. **Repeat** Step 2 until no new pairs are marked.
4. **Result:**
All unmarked pairs are **equivalent**.

Detailed Example: Table-Filling Algorithm

Consider the following DFA,



Following is the transition table for the above DFA

States	Inputs	
	0	1
Q0	Q1	Q2
Q1	Q3	Q4
Q2	Q4	Q3
Q3	Q5	Q5
Q4	Q5	Q5
Q5	Q5	Q5

Minimizing the above DFA using Myhill-Nerode Theorem:

Step 1: Create the pairs of all the states involved in DFA.

	Q0	Q1	Q2	Q3	Q4	Q5
Q0						
Q1						
Q2						
Q3						
Q4						
Q5						

Step 2: Mark all the pairs (Q_a, Q_b) such a that Q_a is Final state and Q_b is Non-Final State.

	Q0	Q1	Q2	Q3	Q4	Q5
Q0						
Q1	✓					
Q2	✓					
Q3		✓	✓			
Q4		✓	✓			
Q5	✓			✓	✓	

Step 3: If there is any unmarked pair (Q_a, Q_b) such a that $\delta(Q_a, x)$ and $\delta(Q_b, x)$ is marked, then mark (Q_a, Q_b) . Here x is a input symbol. Repeat this step until no more marking can be made.

- Check for the unmarked pair Q2, Q1
 - Check when $x=0$: $\delta(Q2, 0) = Q4$ and $\delta(Q1, 0) = Q3$, check if the pair Q4, Q3 is marked and no it is not marked.
 - Check when $x=1$: $\delta(Q2, 1) = Q3$ and $\delta(Q1, 1) = Q4$, check if the pair Q4, Q3 is marked and no it is not marked.
 - Hence we cannot mark the pair Q2, Q1.
- Check for the unmarked pair Q3, Q0
 - Check when $x=0$: $\delta(Q3, 0) = Q5$ and $\delta(Q0, 0) = Q1$, check if the pair Q5, Q1 is marked and no it is not marked.

- Check when $x=1$: $\delta(Q3, 1) = Q5$ and $\delta(Q0, 1) = Q2$, check if the pair Q5, Q2 is marked and no it is not marked.
- Hence we cannot mark the pair Q3, Q0.
- Check for the unmarked pair Q4, Q0
 - Check when $x=0$: $\delta(Q4, 0) = Q5$ and $\delta(Q0, 0) = Q1$, check if the pair Q5, Q1 is marked and no it is not marked.
 - Check when $x=1$: $\delta(Q4, 1) = Q5$ and $\delta(Q0, 1) = Q2$, check if the pair Q5, Q2 is marked and no it is not marked.
 - Hence we cannot mark the pair Q4, Q0.
- Check for the unmarked pair Q4, Q3
 - Check when $x=0$: $\delta(Q4, 0) = Q5$ and $\delta(Q3, 0) = Q5$, Such pair of state Q5, Q5 don't exists.
 - Check when $x=1$: $\delta(Q4, 1) = Q5$ and $\delta(Q3, 1) = Q5$, Such pair of state Q5, Q5 don't exists.
 - Hence we cannot mark the pair Q4, Q3.
- Check for the unmarked pair Q5, Q1
 - Check when $x=0$: $\delta(Q5, 0) = Q5$ and $\delta(Q1, 0) = Q3$, check if the pair Q5, Q3 is marked and yes it is marked.
 - Hence we can mark the pair Q5, Q1.

	Q0	Q1	Q2	Q3	Q4	Q5
Q0						
Q1	✓					
Q2	✓					
Q3		✓	✓			
Q4		✓	✓			
Q5	✓	✓		✓	✓	

- Check for the unmarked pair Q5, Q2
 - Check when $x=0$: $\delta(Q5, 0) = Q5$ and $\delta(Q2, 0) = Q4$, check if the pair Q5, Q4 is marked and yes it is marked.
 - Hence we can mark the pair Q5, Q2.

	Q0	Q1	Q2	Q3	Q4	Q5
Q0						
Q1	✓					
Q2	✓					
Q3		✓	✓			
Q4		✓	✓			
Q5	✓	✓	✓	✓	✓	

- We have checked for all the unmarked pairs but don't need to stop here we need to continue this process until no more markings can be made.
- Check for the unmarked pair Q2, Q1
 - Check when $x=0$: $\delta(Q2, 0) = Q4$ and $\delta(Q1, 0) = Q3$, check if the pair Q4, Q3 is marked and no it is not marked.
 - Check when $x=1$: $\delta(Q2, 1) = Q3$ and $\delta(Q1, 1) = Q4$, check if the pair Q4, Q3 is marked and no it is not marked.
 - Hence we cannot mark the pair Q2, Q1.
- Check for the unmarked pair Q3, Q0
 - Check when $x=0$: $\delta(Q3, 0) = Q5$ and $\delta(Q0, 0) = Q1$, check if the pair Q5, Q1 is marked and yes it is marked.
 - Hence we can mark the pair Q3, Q0.

	Q0	Q1	Q2	Q3	Q4	Q5
Q0						
Q1	✓					
Q2	✓					
Q3	✓	✓	✓			
Q4		✓	✓			
Q5	✓	✓	✓	✓	✓	

- Check for the unmarked pair Q4, Q0
 - Check when $x=0$: $\delta(Q4, 0) = Q5$ and $\delta(Q0, 0) = Q1$, check if the pair Q5, Q1 is marked and yes it is marked.

- Hence we cannot mark the pair Q4, Q0.

	Q0	Q1	Q2	Q3	Q4	Q5
Q0						
Q1	✓					
Q2	✓					
Q3	✓	✓	✓			
Q4	✓	✓	✓			
Q5	✓	✓	✓	✓	✓	

- Check for the unmarked pair Q4, Q3
 - Check when $x=0$: $\delta(Q4, 0) = Q5$ and $\delta(Q3, 0) = Q5$, Such pair of state Q5, Q5 don't exists.
 - Check when $x=1$: $\delta(Q4, 1) = Q5$ and $\delta(Q3, 1) = Q5$, Such pair of state Q5, Q5 don't exists.
 - Hence we cannot mark the pair Q4, Q3.
- Now even though we repeat the procedure we cannot mark the pairs Q2, Q1(since Q4, Q3 is not marked) and Q4, Q3(since Q5, Q5 such pair of states does not exists.). Hence we stop here.

Step 4: Combine all the unmarked pairs and make them as a single state in the minimized DFA.

- The unmarked Pairs are Q2, Q1 and Q4, Q3 hence we combine them.

Following is the Minimized DFA with Q1Q2 and Q3Q4 as the combined states.



- Q0 remains as our starting state.
- Q1 and Q2 were our final states so even we combine them they will remain as the combined final state.
- Q5 is the another final state we have.